TO: Professor Joshua Hertz, Teaching Professor Northeastern University
FROM: Ludovica Melodia, Evan Chu, Hanna Vojar, Northeastern University Class of 2029
DATE: 9 December 2025
SUBJECT: Cornerstone of Engineering: Design Project 2 Memo

Introduction and Context

The purpose of this memo is to detail our solution to Petru's request for a game design. In addition to being the client, Petru is also the intended user of the game, along with a small group of friends. This document outlines the required functions, constraints, and most important traits required for the design of Petru's game. It also includes a description of the final product, a copy of all code used in the design, and a tracker of expenses and time of the design team.

Problem Definition

The intended user of this game is Petru, a second-year Civil Engineering from Northeastern who is particularly interested in transportation engineering. Petru has specifically requested a custom game because he is dissatisfied with the current available solutions. He would like a game that is more interactive than a simple board game, but that is not overcome by the electronic components to become overly technological and "video game" esque. The gameplay should also be strategic rather than random-based.

Because of his academic focus, Petru is interested in all forms of transportation, especially trains and bikes, correlating to his second area of interest, the Netherlands, an area that he has expressed interest in. The game sessions must be relatively short, lasting within 10 to 15 minutes in order to accommodate Petru and his friends' busy schedules. The game must also be transportable so that Petru can carry it between different locations (such as dorm rooms, common areas, or apartments) and durable enough to withstand repeated use by multiple players over time.

The design team consists of first-year engineering students with basic electronics and coding knowledge, access to makerspace facilities, and a budget constraint of $100 maximum for all materials. The game must be designed and fabricated within an eight-week timeframe as part of the GE 1501 course requirements. These resource limitations mean that the solution must be achievable with relatively simple materials while still delivering a unique and engaging experience that justifies Petru's investment in requesting a custom game rather than purchasing one that already exists.

Functions

Successful solutions must include the following functions:

- **Sense player inputs:** The players should be able to accurately interact with the board in order to control the timed aspect, such as when to begin and end the game.

- **Indicate time constraint:** The game must have a way of limiting the length of each player's turn in order to remain within time constraints. This can be shown electronically through timed code, or through a physical timing object.
- **Tracking progress:** Because this game should have a competitive nature, there must be a way for each team's progress to be tracked, by moving through the board or through other scorekeeping components.
- **Indicate turn order**: The electronics must clearly communicate which player should act at any given time. This could be accomplished through LED lights, a physical turn marker, audio cues, or an LCD screen.
- **Generate randomized outcomes**: The game must produce different orders of events so that it can be replayed multiple times by the same users. This randomization could come from card draws, dice rolls, electronic random selection, or algorithmic generation.
- **Present questions/challenges**: The game must display or communicate questions or challenges related to Netherlands geography, train systems, bicycle culture, or engineering concepts. This function engages Petru's specific interests and adds knowledge-based challenges to the strategic elements.

Constraints

- **Time Constraints**: A round of the game must be completed within 10-15 minutes due to the full schedules of Petru and other college students. If the game is too short, the player will feel dissatisfied, and if it runs too long, it will lose priority over academic responsibilities and other activities.
- **Number of Players**: The game must accommodate 2-6 players. The game must remain balanced and engaging for all numbers of players within this game, including uneven amounts.
- **Budget**: Total material costs cannot exceed $100, because the design team consists of college students. The budget must cover all physical components, electronic components, fabrication materials, and any necessary tools or supplies not already available in the maker space
- **Portability**: The complete game system must be easily transportable by one person. This constraint addresses Petru's need to move the game between different playing locations. The maximum practical weight is approximately 20-25 pounds, and the maximum dimensions must allow the game to pass through standard doorways and fit in typical storage spaces such as a backpack.
- **Durability**: The components of this game must withstand repeated use as Petru has requested a replayable game. Because this game is custom-made, it will require investment from the client, so it must provide appropriate value through extended use in order to justify this investment. Components must resist wear from handling, survive minor drops or impacts, and maintain functionality despite environmental variations (temperature, humidity) in typical indoor settings

- **Fabrication Constraints**: The design must be achievable with equipment available in the Northeastern makerspace and skills accessible to first-year engineering students. This includes 3D printers, laser cutters, soldering equipment, the Pico kits and their standard components.
- **Timeline**: Complete design, fabrication, and testing must occur within eight weeks. This constraint affects the complexity of the components that can be included and the amount of testing and design iteration possible
- **Safety**: The game must not present electrical shock hazards, fire risks, sharp edges, or other dangers to players. This constraint applies to all physical and electronic components
- **Theme Specificity**: The game must incorporate transportation themes, particularly trains and bicycles, in ways that would be recognizable, meaningful and entertaining to someone from the Netherlands.

Most Important Traits

Solutions should exhibit these 6 traits, ranked in order of importance:

1. Portability
2. Durability
3. Engaging
4. Balanced Gameplay
5. Replayability
6. Visually appealing

	Portable	**Engaging**	**Durable**	**Balanced**	**Replayable**	**Visually Appealing**	**Total**
Portable	**X**	**1**	**1**	**1**	**1**	**1**	**5**
Engaging	**0**	**X**	**0**	**1**	**1**	**1**	**3**
Durable	**0**	**1**	**X**	**1**	**1**	**1**	**4**
Balanced	**0**	**0**	**0**	**X**	**1**	**1**	**2**
Replayable	**0**	**0**	**0**	**0**	**X**	**1**	**1**
Visually Appealing	**0**	**0**	**0**	**0**	**0**	**X**	**0**

Description of Final Product

The Train Game is a board game with a central electronics box made to enhance gameplay. The physical components consist of a foldable painted chipboard playing board with plastic laser-cut train tokens, four decks of laminated cardstock trivia cards, and a wooden laser-cut box. The

electronic components include three large buttons, six large LED lights to represent 2 traffic lights, a LED strip, and an LCD screen.

The gameplay runs as follows: once the “start” button is pressed, each team has 45 seconds to answer as many trivia questions as possible, which are read by the opposing team. The LED strip in the middle randomizes a color which represents the deck of question cards which each team must answer. The LCD screen and traffic lights serve as visual aids for the timer, allowing the players to keep track of their turns.

The code begins by checking whether the user has pressed the start button. Once the start button has been pressed, the LCD screen shows introductory text followed by the start of a forty-five second round for player one. The pico chooses a random color from four options, indicating which trivia deck to choose from. The code then displays a forty-five second timer on the display and the green LED for player one turns on. When there is 15 seconds left, the yellow LED for player one turns on. When the time is up, the red LED turns on. While this timer is running, the code is also checking whether the player presses the yellow reset button which would then reset the timer. After the timer is up and the round is over, the code asks the user whether they would like to continue. Pressing the white finish button ends the game and turns everything off while the blue button begins the next round for the opposite player. The game then repeats all of the previous steps, starting from choosing a random color. The code alternates rounds between player one and two until they decide to end the game.

Time Sheet/Expense Sheet

Budget Tracker	
Item(s)	Price
Chipboard	$4.33
Paint	$10.99
Paintbrushes	$2.62
Wood for laser cutting	$4.11
Duct Tape	$6.29
Archive Box	$31.54
Total	59.88

Timesheet		
Date	Duration (hours)	Members
10/15	1	All
10/26	2	All
11/10	2	All
11/17	2	Evan
11/18	4	Evan
11/18	3	Hanna
11/19	2.5	All
11/30	4	Ludo
12/1	1	Evan
12/3	2	Ludo
Total	23.5	

Conclusion

The Train Game successfully addresses Petru's request for a custom board game that balances traditional gameplay with electronic components. By incorporating transportation-themed trivia, time management, and visual feedback through LED lights and an LCD display, the final design creates a competitive and engaging experience that aligns with Petru's interests in transportation engineering and Dutch culture.

The current design effectively accommodates 2-6 players within the desired 10 to 15 minute timeframe and provides the replayability requested by Petru through randomized deck selection and a variety of trivia questions. The electronic components enhance gameplay without overwhelming the board game experience, maintaining the balance that Petru requested.

Two areas present opportunities for future improvement. First, portability could be enhanced with design modifications, such as a larger storage box to better organize all components or a folding board mechanism to reduce the overall size of the game. This would make the game even easier for Petru to carry between dorm rooms, common areas, and apartments. Second, additional electronic inputs could increase interactivity, such as buzzers for players to input their own answers, sensors to detect token movement on the board, or audio outputs for sound effects. More cards could also be added for increased playability.

Overall, the Train Game represents a successful custom solution that delivers value to Petru beyond what commercially available games could offer, justifying his investment in a custom-designed product while remaining within the practical constraints of the project.

Code

```
##########################################
#  train_game.py                         #
#                                        #
#                                        #
#                                        #
#    by: Chu                             #
#                                        #
#                                        #
#                                        #
#    A program where the players press   #
#    the blue button to start the game.  #
#    The program alternates between      #
#    timed rounds for each player where  #
#    they answer trivia questions and    #
#    move their piece on a real board.   #
#    The program randomly selects a      #
#    color to indicate which trivia to   #
#    answer questions from. The player   #
#    who reaches the end of the board    #
#    first wins! Who will come out       #
#    victorious in the Train Game!       #
#                                        #
#                                        #
```

```
########################################

# import libraries for pico breadboard, lcd screen, random,
time, and led strip
from machine import I2C, Pin
from pico_i2c_lcd import I2cLcd
from time import sleep
import random
import neopixel


# variable to tell if the game has started
game_start = False

# variable so that the game runs in a continuous loop
game = True

# variable so that the game can pause and make sure the player
is ready
pause = True


# variables for each button
# blue is the start/continue button
# yellow is the reset timer button
# white is the finish button
blue_button = machine.Pin(18, machine.Pin.IN,
machine.Pin.PULL_UP)
yellow_button = machine.Pin(27, machine.Pin.IN,
machine.Pin.PULL_UP)
white_button = machine.Pin(9, machine.Pin.IN,
machine.Pin.PULL_UP)


# variables for all the led lights
red1_led = Pin(14, Pin.OUT)
red2_led = Pin(15, Pin.OUT)
yellow1_led = Pin(12, Pin.OUT)
yellow2_led = Pin(13, Pin.OUT)
green1_led = Pin(16, Pin.OUT)
green2_led = Pin(17, Pin.OUT)
```

```
# variable for the rgb strip
np = neopixel.NeoPixel(machine.Pin(28), 8)


# set up for the lcd screen
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
I2C_ADDR = i2c.scan()[0]
lcd = I2cLcd(i2c, I2C_ADDR, 2, 16)


# allows strings to be typed on the lcd screen
def slowType(lcd, text, delay=0.1):
    for i in text:
        lcd.putstr(i)
        sleep(delay)


# animates the lcd screen to make the text look like it is being
typed out
def write_lcd(text):
    lcd.show_cursor()
    lcd.blink_cursor_on()
    slowType(lcd, str(text), 0.1)
    lcd.blink_cursor_off()
    lcd.hide_cursor()
    sleep(1)
    lcd.clear()


# runs a full 45 second round for player one
def run_round_player1():
    red2_led.on()
    red1_led.on()
    sleep(1)
    count_down = 3
    round_time = 45
    # lcd screen counts down from 3 to begin the round
    for i in range (3):
        write_lcd(str(count_down) + "!")
```

```
        count_down -= 1
    red1_led.off()
    green1_led.on()
    write_lcd("GO!")
    # the round starts and the lcd screen counts down from 45
    for i in range (45):
        write_lcd(round_time)
        round_time -= 1
        # yellow led turns on when there is 15 seconds left
        if i >= 29:
            green1_led.off()
            yellow1_led.on()
        # red led turns on if there is no time left
        if i == 44:
            yellow1_led.off()
            red1_led.on()
        # checks if yellow button is pressed to reset round
timer
        if yellow_button.value() == 0:
            all_leds_off()
            write_lcd("Resetting\ntimer...")
            write_lcd("Are you ready?")
            run_round_player1()
            return
    write_lcd("Stop!")


# runs a full 45 second round for player two
def run_round_player2():
    red1_led.on()
    red2_led.on()
    sleep(1)
    count_down = 3
    round_time = 45
    # lcd screen counts down from 3 to begin the round
    for i in range (3):
        write_lcd(str(count_down) + "!")
        count_down -= 1
    red2_led.off()
    green2_led.on()
    write_lcd("GO!")
```

```
    # the round starts and the lcd screen counts down from 45
    for i in range (45):
        write_lcd(round_time)
        round_time -= 1
        # yellow led turns on when there is 15 seconds left
        if i >= 29:
            green2_led.off()
            yellow2_led.on()
        # red led turns on if there is no time left
        if i == 44:
            yellow2_led.off()
            red2_led.on()
        # checks if yellow button is pressed to reset round
timer
        if yellow_button.value() == 0:
            all_leds_off()
            write_lcd("Resetting\ntimer...")
            write_lcd("Are you ready?")
            run_round_player2()
            return
    write_lcd("Stop!")


# turns all led lights off
def all_leds_off():
    red1_led.off()
    red2_led.off()
    yellow1_led.off()
    yellow2_led.off()
    green1_led.off()
    green2_led.off()


# turns all led lights on
def all_leds_on():
    red1_led.on()
    red2_led.on()
    yellow1_led.on()
    yellow2_led.on()
    green1_led.on()
    green2_led.on()
```

```
# turns the rgb strip a random color out of four colors
def pick_rgb_color():
    r = 0
    g = 0
    b = 0
    roll = random.randint(1, 4)
    if roll == 1:
        r = 255
    elif roll == 2:
        r = 255
        g = 255
    elif roll == 3:
        g = 255
        b = 25
    else:
        b = 255
    for i in range(8):
        np[i] = (r, g, b)
    np.write()


# resets the rgb strip so that it is off
def reset_rgb():
    for i in range(8):
        np[i] = (0, 0, 0)
    np.write()


# turns on all leds and displays the starting messages
def initialize():
    all_leds_on()
    write_lcd("Welcome to the\ntrain game!")
    write_lcd("Are you ready?")


# makes the rgb strip display eight different colors so it looks
like a rainbow
def rainbow_rgb():
    np[0] = (255, 0, 0)
```

```
    np[1] = (255, 165, 0)
    np[2] = (255, 255, 0)
    np[3] = (0, 255, 25)
    np[4] = (0, 0, 255)
    np[5] = (128, 0, 128)
    np[6] = (255, 0, 255)
    np[7] = (255, 255, 255)
    np.write()


####################
# start of the code #
####################


# runs the entire game in an infinite loop so it can be played
an infinite number of times
while game:
    # turns of all led lights and the rgb strip
    reset_rgb()
    all_leds_off()


    # loop that runs and checks if the user presses the start
button to start the game
    while game_start == False:
        if blue_button.value() == 0:
            game_start = True


    # turns on the led lights, turns the rgb strip rainbow, and
displays the starting messages
    rainbow_rgb()
    initialize()


    # loop that runs when the game has started
    while game_start:
        # picks a random color and runs a full round for player
one
        write_lcd("Here is your\ncolor player 1!")
```

```
        pick_rgb_color()
        sleep(1)
        all_leds_off()
        run_round_player1()
        sleep(1)
        rainbow_rgb()
        # pauses the game and checks if the user wants to
continue or end the game
        while pause:
            write_lcd("Do you want\nto continue?")
            if white_button.value() == 0:
                game_start = False
                pause = False
            elif blue_button.value() == 0:
                pause = False
        pause = True

        # checks if the user has continued the game
        if game_start:
            #runs a full round for player two
            write_lcd("Here is your\ncolor player 2!")
            pick_rgb_color()
            sleep(1)
            all_leds_off()
            run_round_player2()
            sleep(1)
            rainbow_rgb()
            # pauses the game and checks if the user wants to
continue or end the game
            while pause:
                write_lcd("Do you want\nto continue?")
                if white_button.value() == 0:
                    game_start = False
                    pause = False
                elif blue_button.value() == 0:
                    pause = False
            pause = True

    # once the user ends the game, turns everything off and
displays a finish message
    all_leds_off()
```

```
reset_rgb()
write_lcd("Game Finished!")
```